Vue.js — Aula 7: Pinia e Projeto Final

Curso de Front-End — Módulo Vue.js · Apostila do aluno

1. Por que Pinia?

Até agora, `props` e `emits` resolveram a comunicação entre **pai e filho**. Mas e quando dois componentes **distantes** na árvore precisam do mesmo dado?

Sem estado compartilhado:

- Cada componente no meio do caminho precisa repassar a prop. Isso se chama **prop drilling** e piora conforme a árvore cresce.
- Componentes irmãos não conseguem conversar diretamente.
- Duplicar a mesma lista em telas diferentes gera dados desincronizados.

Com Pinia:

- O estado compartilhado mora em um lugar só: a **store**.
- Qualquer componente lê ou altera a store diretamente, sem repassar props.
- Continua reativo: mudou na store, a tela atualiza sozinha.

> **Pinia** é a biblioteca oficial de gerenciamento de estado do Vue, sucessora do Vuex.

2. Instalando e conectando o Pinia

Assim como o router, o Pinia é conectado à aplicação **uma vez**, no `main.js`.

```
npm install pinia
```

```
// src/main.js
import { createApp } from "vue";
import { createPinia } from "pinia";
import App from "./App.vue";
import router from "./router";

const app = createApp(App);
app.use(router);
app.use(createPinia());
app.mount("#app");
```

3. Criando uma store: state, getters e actions

Uma store tem três partes, muito parecidas com o que você já conhece de um componente:

Parte da store	Equivale a...	Para que serve
`state`	`data()`	Os dados guardados na store
`getters`	`computed`	Valores calculados a partir do state
`actions`	`methods`	Funções que alteram o state

> Se você entendeu `data()`, `computed` e `methods`, já entendeu 90% do Pinia. Só muda onde essas peças moram.

```
// src/stores/tarefas.js
import { defineStore } from "pinia";

export const useTarefasStore = defineStore("tarefas", {
  state: () => ({
    lista: []
  }),
  getters: {
    restantes: (state) => state.lista.filter((t) => !t.feita).length
  },
  actions: {
    adicionar(texto) {
      this.lista.push({ id: Date.now(), texto, feita: false });
    },
    marcarFeita(id) {
      const tarefa = this.lista.find((t) => t.id === id);
      tarefa.feita = true;
    }
  }
});
```

`defineStore("tarefas", ...)` cria uma store com o nome `"tarefas"`. Esse nome identifica a store internamente no Pinia. Por convenção, a função exportada começa com `use` e termina com `Store`.

Dentro das actions, usamos `this` para acessar o state, como em um `method` comum.

4. Usando a store em um componente

Qualquer componente pode importar a store, guardá-la em `data()` e usá-la no template.

```
<!-- Tarefas.vue -->
<template>
  <p>{{ store.restantes }} restantes</p>
  <ul>
    <li v-for="t in store.lista" :key="t.id">
      {{ t.texto }}
      <button @click="store.marcarFeita(t.id)">Concluir</button>
    </li>
  </ul>
</template>

<script>
import { useTarefasStore } from "../stores/tarefas";

export default {
  data() {
    return { store: useTarefasStore() };
  }
};
</script>
```

Repare:

- `useTarefasStore()` devolve a **mesma instância** em qualquer componente que chamar essa função.
- `store.lista` e `store.restantes` funcionam como qualquer dado reativo no template.
- Chamar uma action (`store.marcarFeita(id)`) é como chamar um method comum.

5. Um estado, vários componentes

Diferente de props e emits, a store **não segue a árvore de componentes**. `Header.vue`, `Tarefas.vue` e `Rodape.vue` não têm relação de pai e filho entre si, mas todos podem ler e alterar a mesma store, sem passar uma única prop.

6. Projeto final: lista de tarefas completa

O projeto junta as peças de todas as aulas:

1. **App.vue + Router:** navegação entre Home, Tarefas e Sobre (Aula 5)

2. **Store de tarefas:** state, getters e actions compartilhados (hoje)
3. **Busca inicial na API:** `created()` assíncrono preenche a store (Aula 6)
4. **TarefaItem.vue:** componente filho com props e emits (Aula 3)
5. **Ciclo de vida e watch:** logs e reações a mudanças de estado (Aula 4)

Componente filho do projeto:

```
<!-- TarefaItem.vue -->
<template>
  <li>
    {{ texto }}
    <button @click="$emit('concluir')">Concluir</button>
  </li>
</template>

<script>
export default {
  props: ["texto"],
  emits: ["concluir"]
};
</script>
```

7. Resumo rápido

- **Pinia** guarda estado compartilhado entre componentes, sem prop drilling.
- `app.use(createPinia())` no `main.js` conecta o Pinia uma vez.
- `defineStore("nome", { state, getters, actions })` cria a store.
- `state` = `data()`, `getters` = `computed`, `actions` = `methods`.
- `useXStore()` devolve a mesma store em qualquer componente.

8. Exercícios práticos

Use o projeto das aulas anteriores (Vite + Vue Router) como base.

Exercício 1: Instalando o Pinia

No terminal do seu projeto, instale o Pinia. Depois, abra o `src/main.js`, importe `createPinia` e conecte-o à aplicação com `app.use(...)`. Rode o projeto e confirme que não aparece nenhum erro no console.

Exercício 2: Minha primeira store

Crie o arquivo `src/stores/tarefas.js` com `defineStore("tarefas", ...)` e um `state` com a propriedade `lista`, começando como um array vazio. Exporte a store com o nome `useTarefasStore`.

Exercício 3: Preenchendo o state

Na store do exercício 2, comece a `lista` com 3 tarefas fixas, no formato `{ id: 1, texto: "Estudar Pinia", feita: false }`. Em `Tarefas.vue`, importe a store, guarde-a em `data()` e mostre os textos das tarefas com `v-for` e `:key`.

Exercício 4: Criando um getter

Adicione à store um getter chamado `restantes`, que retorna quantas tarefas ainda não foram feitas. Mostre no template de `Tarefas.vue` a mensagem: **"X tarefas restantes"**.

Exercício 5: Action para marcar como feita

Crie a action `marcarFeita(id)`, que encontra a tarefa pelo `id` e muda `feita` para `true`. Adicione um botão **Concluir** em cada tarefa que chama a action. Observe o contador de restantes diminuir sozinho.

Exercício 6: Action para adicionar

Crie a action `adicionar(texto)`, que coloca uma nova tarefa no fim da lista com `id: Date.now()` e `feita: false`. Em `Tarefas.vue`, crie um campo de texto com `v-model` e um botão **Adicionar** que chama a action e limpa o campo depois.

Exercício 7: Dois componentes, uma store

Crie dois componentes novos, `Header.vue` e `Rodape.vue`, sem nenhuma relação de pai e filho com `Tarefas.vue`. O `Header` deve mostrar o número de tarefas restantes. O `Rodape` deve mostrar o total de tarefas na lista. Use apenas a store, **sem passar props**. Adicione uma tarefa em `Tarefas.vue` e confirme que os três componentes atualizam juntos.

Exercício 8: Componente filho com a store

Crie o `TarefaItem.vue` com a prop `texto` e o emit `concluir`, como no exemplo da apostila. Em `Tarefas.vue`, substitua o `<li>` por `<TarefaItem>`, passando `:texto="t.texto"`. Quando o filho emitir `concluir`, chame `store.marcarFeita(t.id)` no pai.

Exercício 9: Tarefas vindas de uma API

Em `Tarefas.vue`, crie um `created()` assíncrono que busca as tarefas em `https://jsonplaceholder.typicode.com/todos?_limit=5` usando `fetch` e `async/await`. Converta cada item para o formato da store (`title` vira `texto`, `completed` vira `feita`) e salve o resultado em `store.lista`, **não** em um `data()` local. Use `try/catch` e as variáveis `carregando` e `erro` para mostrar "Carregando..." ou uma mensagem de erro na tela.

Exercício 10: Projeto final completo

Junte tudo em uma aplicação só:

- Rotas para **Home**, **Tarefas** e **Sobre**, com `router-link` no menu
- `Header.vue` aparecendo em todas as telas, mostrando as tarefas restantes
- Tarefas iniciais vindas da API, guardadas na store
- Cada tarefa renderizada com `TarefaItem.vue` (props e emits)
- Um `watch` em `store.restantes` que mostra no console uma mensagem quando todas as tarefas forem concluídas

Navegue entre as páginas e confirme que o estado da lista **não se perde** ao trocar de rota.

Para continuar evoluindo

- **Composition API** (`script setup`): outra forma de organizar a lógica dos componentes
- **TypeScript:** tipagem para projetos maiores
- **Nuxt:** framework Vue com renderização no servidor
- **Vitest:** testes automatizados

Parabéns por concluir o módulo de Vue.js!